

Tutor

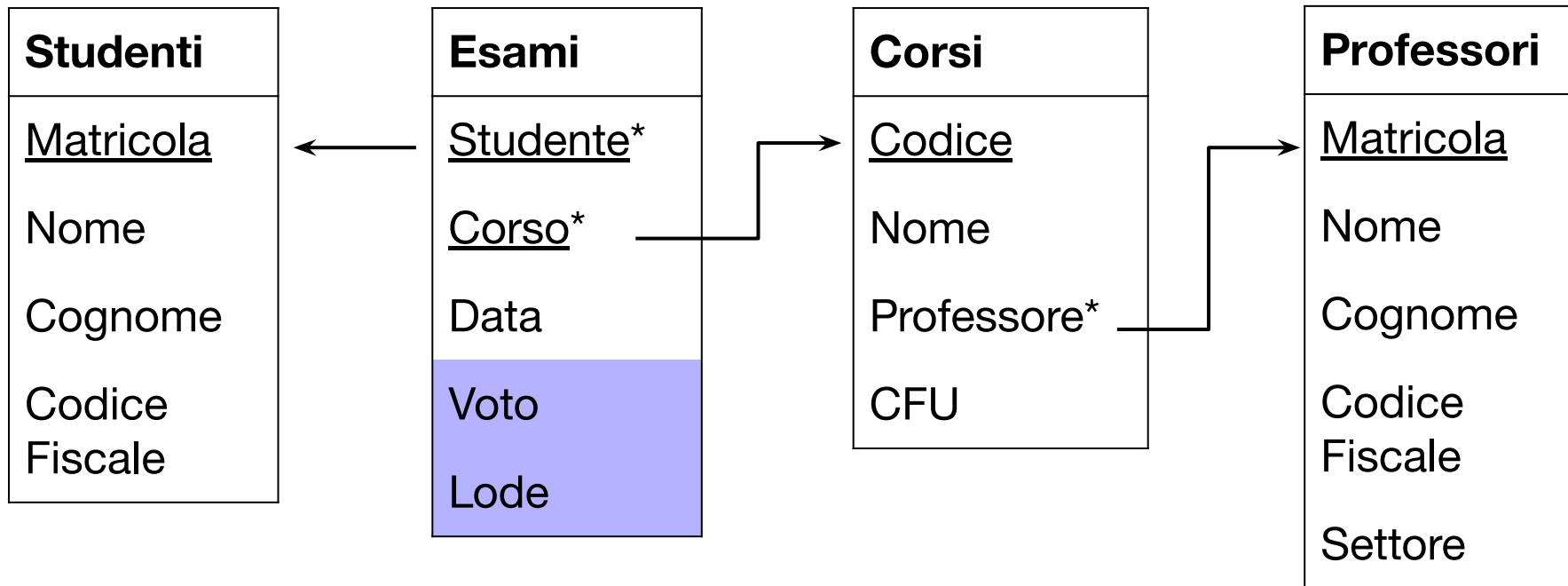
Mauro Farina

- Dottorando in ingegneria industriale e dell'informazione
- 2° anno
- Misure di Internet
- mauro.farina@phd.units.it

Repository esercitazioni

- github.com/mauro-farina/database-2026

Database



Esercizio 5: Errata corrige

La data più recente in cui è stato sostenuto un esame di ciascun professore

```
SELECT
    concat("Prof. ", p.nome, " ", p.cognome) AS prof,
    max(e.data) AS ultimo_esame
FROM professori p
LEFT JOIN corsi c ON c.professore = p.matricola
LEFT JOIN esami e ON e.corso = c.codice
GROUP BY p.matricola;
```

```
SELECT ...  
FROM professori prof  
LEFT JOIN corsi ...  
LEFT JOIN esami ...  
GROUP BY prof.matricola
```



p.matricola	p.nome	p.cognome	esami.data
1	Andrea	De Lorenzo	2022-01-11
			2023-09-23
			2021-06-18
2	Eric	Medvet	2017-02-08
			2019-01-28

```
SELECT ...  
FROM professori prof  
LEFT JOIN corsi ...  
LEFT JOIN esami ...  
GROUP BY prof.matricola
```



p.matricola	p.nome	p.cognome	esami.data
1	Andrea	De Lorenzo	2022-01-11
	Andrea	De Lorenzo	2023-09-23
	Andrea	De Lorenzo	2021-06-18
2	Eric	Medvet	2017-02-08
	Eric	Medvet	2019-01-28

```
SELECT p.nome, p.cognome
FROM professori prof
LEFT JOIN corsi ...
LEFT JOIN esami ...
GROUP BY prof.matricola
```

→ **Colonne non aggregate**

p.matricola	p.nome	p.cognome	esami.data
1	Andrea	De Lorenzo	2022-01-11
	Andrea	De Lorenzo	2023-09-23
	Andrea	De Lorenzo	2021-06-18
2	Eric	Medvet	2017-02-08
	Eric	Medvet	2019-01-28

```
SELECT p.nome, p.cognome
FROM professori prof
LEFT JOIN corsi ...
LEFT JOIN esami ...
GROUP BY prof.matricola
```

In base al database (e alla versione utilizzata):

- Seleziona uno a caso
- Restituisce errore

p.matricola	p.nome	p.cognome	esami.data
1	Andrea	De Lorenzo	2022-01-11
	Andrea	De Lorenzo	2023-09-23
	Andrea	De Lorenzo	2021-06-18
2	Eric	Medvet	2017-02-08
	Eric	Medvet	2019-01-28

Esercizio 5: Soluzione

È buona prassi che le colonne selezionate siano chiavi del group by o colonne aggregate

```
SELECT
    concat("Prof. ", p.nome, " ", p.cognome) AS prof,
    max(e.data) AS ultimo_esame
FROM professori p
LEFT JOIN corsi c ON c.professore = p.matricola
LEFT JOIN esami e ON e.corso = c.codice
GROUP BY s.matricola, s.nome, s.cognome;
```


Esercizio 5: Altre soluzioni (1)

La data più recente in cui è stato sostenuto un esame di ciascun professore

→ Raggruppa **professori con lo stesso nome / cognome**

```
SELECT
    concat("Prof. ", p.nome, " ", p.cognome) AS prof,
    max(e.data) AS ultimo_esame
FROM professori p
LEFT JOIN corsi c ON c.professore = p.matricola
LEFT JOIN esami e ON e.corso = c.codice
GROUP BY p.nome, p.cognome;
```

Esercizio 5: Altre soluzioni (2)

La data più recente in cui è stato sostenuto un esame di ciascun professore

→ Raggruppa **professori con lo stesso nome / cognome**

```
SELECT
    concat("Prof. ", p.nome, " ", p.cognome) AS prof,
    max(e.data) AS ultimo_esame
FROM professori p
LEFT JOIN corsi c ON c.professore = p.matricola
LEFT JOIN esami e ON e.corso = c.codice
GROUP BY prof;
```

GROUP BY: singola entità

Media voti per **studente**:

esami	
studente	voto
IN0500853	18
IN0500853	26
SM3200112	23
SM3200112	25
IN0500025	30

```
SELECT studente, avg(voto)
FROM esami
GROUP BY studente;
```

GROUP BY: singola entità

Media voti per **studente**:

esami	
studente	voto
IN0500853	18
IN0500853	26
SM3200112	23
SM3200112	25
IN0500025	30

```
SELECT studente, avg(voto)
FROM esami
GROUP BY studente;
```

studente	voto	avg(voto)
IN0500853	18, 26	22.00
SM3200112	23, 25	24.00
IN0500025	30	30.00

GROUP BY: caratteristiche comuni

Media voti per **corso di laurea**:

esami	
studente	voto
IN0500853	18
IN0500853	26
SM3200112	23
SM3200112	25
IN0500025	30

```
SELECT studente, avg(voto)
FROM esami GROUP BY
substring(studente, 1, 4);
```

GROUP BY: caratteristiche comuni

Media voti per **corso di laurea**:

esami	
studente	voto
IN0500853	18
IN0500853	26
SM3200112	23
SM3200112	25
IN0500025	30

```
SELECT studente, avg(voto)
FROM esami GROUP BY
substring(studente, 1, 4);
```

cdl	voto	avg(voto)
IN05	18, 26, 30	24.67
SM32	23, 25	24.00

Esercizi

Caricare la soluzione (**codice SQL**) sull'apposito Google Form:

- github.com/mauro-farina/database-2026
- forms.gle/X7Zn2fXr1VgLmKyP9

Esercizio 10

Quanti studenti non hanno mai preso 30L?

- Risultato atteso: 98

Diversi modi per risolvere l'esercizio:

- Subquery
- EXISTS
- LEFT OUTER JOIN
- GROUP BY

Esercizio 10: Soluzione (1)

Quanti studenti non hanno mai preso 30L?

- Soluzione con **subquery**

```
SELECT count(*)  
FROM studenti  
WHERE matricola NOT IN (  
    SELECT DISTINCT studente  
    FROM esami  
    WHERE lode IS TRUE  
);
```

Esercizio 10: Soluzione (2)

Quanti studenti non hanno mai preso 30L?

- Soluzione con **EXISTS**

```
SELECT count(*)  
FROM studenti s  
WHERE NOT EXISTS (  
    SELECT * FROM esami e  
    WHERE lode IS TRUE  
        AND s.matricola = e.studente  
);
```

Esercizio 10: Soluzione (3)

Quanti studenti non hanno mai preso 30L?

- Soluzione con `LEFT OUTER JOIN` (1)

```
SELECT count(*)  
FROM studenti s  
LEFT OUTER JOIN (  
    SELECT * FROM esami WHERE lode IS TRUE  
) as tabellaLodi  
ON s.matricola = tabellaLodi.studente  
WHERE tabellaLodi.voto IS NULL;
```

Esercizio 10: Soluzione (3)

Quanti studenti non hanno mai preso 30L?

- Soluzione con `LEFT OUTER JOIN` (1)

```
SELECT count(*)  
FROM studenti s  
LEFT OUTER JOIN (  
    SELECT * FROM esami WHERE lode IS TRUE  
) as tabellaLodi  
ON s.matricola = tabellaLodi.studente  
WHERE tabellaLodi.voto IS NULL;
```

Esercizio 10: Soluzione (3)

Quanti studenti non hanno mai preso 30L?

- Soluzione con `LEFT OUTER JOIN` (2)

```
SELECT count(*)  
FROM studenti s  
LEFT OUTER JOIN esami e  
ON s.matricola = e.studente AND e.lode IS TRUE  
WHERE e.voto IS NULL;
```

Esercizio 10: Soluzione (4)

Quanti studenti non hanno mai preso 30L?

- Soluzione con **GROUP BY**

```
SELECT count(*) FROM (  
    SELECT s.matricola FROM studenti s  
    LEFT OUTER JOIN esami e  
    ON e.studente = s.matricola  
    GROUP BY s.matricola  
    HAVING sum(e.lode) = 0 OR sum(e.lode) IS NULL  
) as tab;
```

Esercizio 11

1. Aggiungere la colonna booleana **buono** (con valore di default false) alla tabella **professori**
2. Assegnare **buono=TRUE** ai professori che hanno dato almeno una lode
 - a. Sarà necessario disabilitare i *safe updates* di MySQL
3. Con una query, restituire il numero di professori "buoni"
 - a. Risultato atteso: 24

Esercizio 11: Soluzione (1)

1. Aggiungere la colonna booleana **buono** (con valore di default false) alla tabella **professori**

```
ALTER TABLE professori  
ADD COLUMN buono BOOL DEFAULT FALSE;
```


Esercizio 11: Soluzione (2)

2. Assegnare `buono=TRUE` ai professori che hanno dato almeno una lode

```
SET SQL_SAFE_UPDATES = 0; -- disabilita safe updates
UPDATE professori SET buono = TRUE
WHERE matricola IN (
    SELECT DISTINCT c.professore FROM corsi c
    JOIN esami e ON e.corso = c.codice
    WHERE e.lode IS TRUE
);
SET SQL_SAFE_UPDATES = 1; -- abilita safe updates
```

Esercizio 11: Soluzione (3)

3. Con una query, restituire il numero di professori "buoni"

```
SELECT count(*)  
FROM professori  
WHERE buono IS TRUE;
```

Esercizio 12

Per ogni studente che ha migliorato la propria media (ponderata) almeno una volta, restituire il numero di volte in cui ciò è avvenuto

- **Non** è richiesto che l'esercizio sia svolto con un'unica query

Risultato atteso (248 righe), in ordine alfabetico di matricola sono:

- EC2100025 6
- EC2100142 5
- EC2100149 8
- ...
- SM3500963 1
- SM3500986 3